Veri Yapıları

Giriş ve C++ Temelleri

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

İçerik-Teorik

- Dersin Tanıtımı ve Hedefleri
- Veri Yapıları ve Algoritmaların Önemi
- Matematiksel Arka Planın Gözden Geçirilmesi
- Özyineleme (Recursion) Kavramına Giriş
- C++ ile Nesneye Yönelik Programlamaya Bakış
  - Sınıflar (Classes)
  - Yapıcı Metotlar (Constructors)
  - Erişim Belirleyiciler (Access Specifiers)
- C++'ın Temel Özellikleri
  - Referanslar ve İşaretçiler (References & Pointers)
  - Parametre Geçirme Yöntemleri

Bu Ders Neden Önemli?

- Yazılım Mühendisliği ve Veri Yapıları:
  - "Her program, verileri işlemek için vardır." ilkesi.
- Problemi Doğru Modelleme:
  - Gerçek dünya problemlerini, bilgisayar ortamında verimli bir şekilde temsil etme sanatı.
- Performans ve Verimlilik:
  - Milyonlarca kullanıcıya hizmet veren bir sosyal medya uygulamasının anlık veri akışını nasıl yönettiğini hiç düşündünüz mü?
  - Bir harita uygulamasının en kısa yolu saniyeler içinde nasıl bulduğunu merak ettiniz mi?
- Doğru Veri Yapısını Seçmek:
  - Bir kelime işlemcideki "geri al" (undo) özelliği nasıl çalışır? (Yığın - Stack)
  - Bir ağdaki en ucuz uçuş rotası nasıl bulunur? (Graf - Graph)

Algoritma Nedir?
Algoritma Analizi Neden Gerekli?

- Algoritma:
  - Belirli bir problemi çözmek için izlenen, sonlu adımlardan oluşan bir yol.
- Algoritma Analizi:
  - Bir algoritmanın ne kadar "iyi" olduğunu ölçmek.
  - Zaman Karmaşıklığı (Time Complexity):
    - Algoritmanın ne kadar sürede çalıştığı.
  - Alan Karmaşıklığı (Space Complexity):
    - Algoritmanın ne kadar bellek kullandığı.
- Neden Önemli?:
  - Donanım hızlansa bile, verimsiz bir algoritma büyük veri setlerinde kabul edilemez sonuçlar doğurur.

Matematiksel Temellerin Gözden Geçirilmesi

- Logaritma (log):
  - Özellikle "böl ve yönet" algoritmalarında (örneğin, ikili arama) karşımıza çıkar. Veri seti her adımda küçüldüğünde logaritmik bir performans elde edilir.
  - Bilgisayar bilimlerinde, aksi belirtilmediği sürece tüm logaritmalar 2 tabanındadır.

$$\log_A B = \frac{\log_C B}{\log_C A}; \quad A, B, C > 0, A \neq 1$$

$$\log AB = \log A + \log B; \quad A, B > 0$$

$$\log A/B = \log A - \log B$$

$$\log(A^B) = B \log A$$

$$\log X < X \quad \text{for all } X > 0$$

$$\log 1 = 0, \quad \log 2 = 1, \quad \log 1{,}024 = 10, \quad \log 1{,}048{,}576 = 20$$

Matematiksel Temellerin Gözden Geçirilmesi

- Seriler ve Toplamlar (∑):
  - Döngülerin maliyetini hesaplarken kullanılır.

$$\sum_{i=0}^{N} 2^i = 2^{N+1} - 1$$

$$\sum_{i=0}^{N} A^i = \frac{A^{N+1} - 1}{A - 1}$$

$$\sum_{i=1}^{N} f(N) = Nf(N)$$

$$\sum_{i=n_0}^{N} f(i) = \sum_{i=1}^{N} f(i) - \sum_{i=1}^{n_0-1} f(i)$$

$$S = 1 + A + A^2 + A^3 + A^4 + A^5 + \cdots$$

$$AS = A + A^2 + A^3 + A^4 + A^5 + \cdots$$

$$S = \frac{1}{2} + \frac{2}{2^2} + \frac{3}{2^3} + \frac{4}{2^4} + \frac{5}{2^5} + \cdots$$

$$S = 2.$$

Matematiksel Temellerin Gözden Geçirilmesi

- Modüler Aritmetik:
  - Hash tabloları gibi veri yapılarında temel bir rol oynar.

$$A \equiv B \pmod{N}$$

$$81 \equiv 61 \equiv 1 \pmod{10}$$

if $A \equiv B \pmod{N}$

$$A + C \equiv B + C \pmod{N}$$

$$AD \equiv BD \pmod{N}$$

Özyineleme (Recursion) Kavramına Giriş

- Tanım:
  - Bir fonksiyonun kendi kendini çağırması.
- Temel Bileşenler:
  - Temel Durum (Base Case):
    - Fonksiyonun kendini çağırmayı durdurduğu nokta.
  - Özyinelemeli Adım (Recursive Step):
    - Problemi daha küçük bir alt probleme indirgeyerek kendini çağırdığı adım.
- Örnek: Faktöriyel Hesabı
  - $n! = n \times (n-1)!$
  - Temel Durum: $0! = 1$

Özyineleme (Recursion) Kavramına Giriş

```
1   int f( int x )
2   {
3       if( x == 0 )
4           return 0;
5       else
6           return 2 * f( x - 1 ) +  x * x;
7   }
```

Figure 1.2 A recursive function

Özyineleme (Recursion) Kavramına Giriş

- Özyinelemenin iki temel kuralı
  - Temel durumlar (Base cases):
    - Her zaman yineleme olmayan bazı temel durumlarınız olmalıdır.
  - İlerleme kaydetme (Making progress):
    - Yinelemeli çağrı her zaman bir temel duruma doğru ilerlemelidir.

```
1   int bad( int n )
2   {
3       if( n == 0 )
4           return 0;
5       else
6           return bad( n / 3 + 1 ) + n - 1;
7   }
```

Figure 1.3 A nonterminating recursive function

Sonlanmayan Kötü Örnek

C++'a Hızlı Bir Bakış: Neden Veri Yapıları İçin C++?

- Performans:
  - C++'ın C temelli olması, donanıma yakın ve hızlı çalışmasını sağlar.
- Nesneye Yönelik Programlama (OOP):
  - Veri ve bu veriyi işleyen fonksiyonları bir arada tutarak (encapsulation) veri yapılarının soyutlanmasını kolaylaştırır.
- STL (Standard Template Library):
  - vector, list, map gibi hazır ve optimize edilmiş veri yapıları sunar.

C++ Temelleri

- https://www.geeksforgeeks.org/cpp/c-plus-plus/
- C++ Fundamentals
  - Introduction, Identifiers, Keywords, Variables, Data Types, Operators, Basic Input/Output, Conditional Statement, Loops
- C++ Function
  - Functions, Default Arguments, Inline Functions, Lambda Expression
- Compound Data Types
  - Arrays, Strings, Structures, Unions, Enumeration
- Ve diğerleri...

Sınıflar (Classes) - Veri Yapılarının Temel Taşı

- Tanım:
  - Bir veri yapısını ve onun üzerinde çalışan metotları (fonksiyonları) bir araya getiren şablon.
- Örnek: Ogrenci Sınıfı
  - Veri Üyeleri (Data Members):
    - ogrenciNo, ad, soyad
  - Metotlar (Member Functions):
    - bilgileriGoster(), notEkle()
- public ve private Erişim Belirleyicileri:
  - Bilgi gizleme (information hiding) prensibi.

Yapıcı (Constructor) ve Yıkıcı (Destructor) Metotlar

- Yapıcı Metot (Constructor):
  - Bir nesne oluşturulduğunda otomatik olarak çağrılır.
  - Başlangıç değerlerini atamak için kullanılır.
- Yıkıcı Metot (Destructor):
  - Bir nesnenin ömrü bittiğinde (örneğin, bellekten silindiğinde) otomatik olarak çağrılır.
  - Genellikle dinamik olarak ayrılan belleği serbest bırakmak için kullanılır.

Arayüz ve İmplementasyonun Ayrılması

- Header Dosyaları (.h veya .hpp):
  - Sınıfın arayüzünü (tanımını) içerir.
  - Ne yaptığıyla ilgilenir.
- Source Dosyaları (.cpp):
  - Metotların gövdelerini (implementasyonunu) içerir.
  - Nasıl yaptığıyla ilgilenir.
- Neden Önemli?:
  - Kodun modülerliğini, okunabilirliğini ve yeniden kullanılabilirliğini artırır.

Arayüz ve İmplementasyonun Ayrılması

```
1  #ifndef IntCell_H
2  #define IntCell_H
3
4  /**
5   * A class for simulating an integer memory cell.
6   */
7  class IntCell
8  {
9    public:
10     explicit IntCell( int initialValue = 0 );
11     int read( ) const;
12     void write( int x );
13
14   private:
15     int storedValue;
16 };
17
18 #endif
```

Figure 1.7 `IntCell` class interface in file *IntCell.h*

```
1  #include "IntCell.h"
2
3  /**
4   * Construct the IntCell with initialValue
5   */
6  IntCell::IntCell( int initialValue ) : storedValue{ initialValue }
7  {
8  }
9
10 /**
11  * Return the stored value.
12  */
13 int IntCell::read( ) const
14 {
15     return storedValue;
16 }
17
18 /**
19  * Store x.
20  */
21 void IntCell::write( int x )
22 {
23     storedValue = x;
24 }
```

Figure 1.8 `IntCell` class implementation in file *IntCell.cpp*

```
1  #include <iostream>
2  #include "IntCell.h"
3  using namespace std;
4
5  int main( )
6  {
7      IntCell m;
8
9      m.write( 5 );
10     cout << "Cell contents: " << m.read( ) << endl;
11
12     return 0;
13 }
```

Figure 1.9 Program that uses `IntCell` in file *TestIntCell.cpp*

Vektörler

```cpp
int main() {
    const std::pmr::vector<int> squares { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    int sum = 0;

    for( int i = 0; i < squares.size( ); ++i )
        sum += squares[ i ];
    std::cout << sum << std::endl;

    sum=0;
    for(const int square : squares)
        sum += square;
    std::cout << sum << std::endl;

    float sum2 = 0;
    for( auto x : squares )
        sum2 += x;
    std::cout << sum2 << std::endl;

    sum2 = 0;
    const std::pmr::vector<float> squares2 { 31.3, 28.3, 31.3, 30.3, 31.3, 30.3, 31.3, 31.3, 30.3, 31.3, 30.3, 31.3 };
    for( auto x : squares2 )
        sum2 += x;
    std::cout << sum2 << std::endl;
}
```

Çıktı

```
365
365
365
368.6
```

C++ Detayları: İşaretçiler (Pointers)

- Tanım:
  - Bellekteki bir adresin tutulduğu değişken.
- Neden Gerekli?:
  - Dinamik bellek yönetimi (new ve delete operatörleri). Bağlı listeler, ağaçlar gibi veri yapılarının temelidir.
  - Büyük nesnelerin fonksiyonlara kopyalanmadan, sadece adresleriyle geçirilmesini sağlayarak performansı artırır.
- Sözdizimi: * (içerik operatörü), & (adres operatörü).

C++ Detayları: İşaretçiler (Pointers)

```
1   int main( )
2   {
3       IntCell *m;
4
5       m = new IntCell{ 0 };
6       m->write( 5 );
7       cout << "Cell contents: " << m->read( ) << endl;
8
9       delete m;
10      return 0;
11  }
```

Figure 1.11 Program that uses pointers to `IntCell` (there is no compelling reason to do this)

- * operatörü ile oluşturulan IntCell nesnesinin hafızada tutulduğu adres bilgisi m değişkenine atanır!
- C++ dilinde Java'da olduğu gibi Çöp Toplayıcısı (Garbage Collector) yoktur.
- delete komutu ile m nesnesi serbest bırakılır böylelikle hafıza taşmasının (memory leak) önüne geçilir

C++ Detayları: Referanslar (References)

- Tanım:
  - Mevcut bir değişken için ikinci bir isim, bir takma ad (alias).
  - & operatörünün ikinci temel kullanımıdır
- İşaretçiden Farkı:
  - Tanımlandığı anda bir değere atanmak zorundadır.
  - null olamaz.
  - Bir kere atandıktan sonra başka bir değişkeni gösteremez.
- Kullanım Alanı:
  - Özellikle fonksiyonlara parametre geçerken güvenli ve verimli bir alternatif sunar.

C++ Detayları: Referanslar (References)

```cpp
using namespace std;

int main() {
    int x = 10;

    int& ref = x;
    cout << ref << endl;

    ref = 22;
    cout << ref << endl;
    cout << x;

    return 0;
}
```

Çıktı

```
10
22
22
```

Parametre Geçirme Yöntemleri

- Değer ile Çağrı (Pass by Value)
- Referans ile Çağrı (Pass by Reference)
- İşaretçi ile Çağrı (Pass by Pointer)

Parametre Geçirme Yöntemleri

- Değer ile Çağrı (Pass by Value):
  - Parametrenin bir kopyası oluşturulur.
  - Fonksiyon içindeki değişiklikler orijinal değişkeni etkilemez.

```cpp
#include <iostream>
using namespace std;

// Arguments are pass by value
void change(int a) {

    // Modifying arguments
    a = 22;
}

int main() {
    int x = 5;

    // Passing x by value to change()
    change(x);

    cout << x;
    return 0;
}
```

Çıktı

5

Parametre Geçirme Yöntemleri

- Referans ile Çağrı (Pass by Reference):
  - Parametrenin kendisi (adresi) gönderilir.
  - Fonksiyon içindeki değişiklikler orijinal değişkeni etkiler.
  - Kopyalama maliyetinden kurtarır.

```cpp
#include <iostream>
using namespace std;

// Arguments are pass by value
void change(int& a) {

    // Modifying arguments
    a = 22;
}

int main() {
    int x = 5;

    // Passing x by reference to change()
    change(x);

    cout << x;
    return 0;
}
```

Çıktı

22

Parametre Geçirme Yöntemleri

- İşaretçi ile Çağrı (Pass by Pointer):
  - Parametrenin adresi bir işaretçi olarak gönderilir.
  - Referansa benzer şekilde orijinal veriyi değiştirme imkanı sunar.

```cpp
#include <iostream>
using namespace std;

// Arguments are pass by value
void change(int* a) {

    // Modifying arguments
    *a = 22;
}

int main() {
    int x = 5;

    // Passing address of x to change()
    change(&x);

    cout << x;
    return 0;
}
```

Çıktı

22

Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

Veri Yapıları

Giriş ve C++ Temelleri
(Uygulama Dersi)

Dr. Öğr. Üyesi Muhammed Ali KOŞAN

İçerik-Uygulama

- Geliştirme Ortamının Kurulumu (IDE, Compiler)
- Temel C++ Programı Yazma ve Derleme
- Sınıf (Class) Tanımlama ve Nesne (Object) Oluşturma
- İşaretçi (Pointer) ve Referans (Reference) Kullanımı
- Basit Bir Veri Yönetim Sınıfı Tasarımı

Ortam Kurulumu ve İlk Program

- IDE Kurulumu:
  - Visual Studio, VS Code (MinGW ile), veya **CLion** gibi bir C++ geliştirme ortamının kurulumu ve konfigürasyonu.
- İlk Program: "Merhaba, Veri Yapıları!“
  - iostream kütüphanesinin kullanımı.
  - main fonksiyonu.
  - cin ile girdi alma ve cout ile ekrana yazı yazdırma.
  - Programı derleme (compile) ve çalıştırma (run) adımları.

Uygulama 1: Dikdortgen Sınıf Tasarımı

1. Dikdortgen.h dosyası oluşturun:
   a. private üyeler: genislik, yukseklik (double tipinde).
   b. public üyeler:
      i. Bir varsayılan yapıcı metot (constructor).
      ii. Genişlik ve yüksekliği parametre alan bir yapıcı metot.
      iii. alanHesapla() metodu.
      iv. cevreHesapla() metodu.
      v. bilgileriYazdir() metodu.
2. Dikdortgen.cpp dosyası oluşturun:
   a. Metotların gövdelerini (implementasyonlarını) yazın.
3. main.cpp dosyası oluşturun:
   a. Dikdortgen sınıfından birkaç nesne (örneğin, d1 ve d2) oluşturun.
   b. Nesnelerin metotlarını çağırarak alan ve çevrelerini hesaplatıp ekrana yazdırın.

Uygulama 2: Değişken Değerlerini Değiştirme (Swap)

1. Bir main fonksiyonu içinde iki int değişken tanımlayın (a = 10, b = 20).
2. swapByValue(int x, int y) fonksiyonunu yazın ve çağırın.
   - Sonucu gözlemleyin (a ve b değişmeyecektir).
3. swapByReference(int &x, int &y) fonksiyonunu yazın ve çağırın.
   - Sonucu gözlemleyin (a ve b yer değiştirecektir).
4. swapByPointer(int *x, int *y) fonksiyonunu yazın ve çağırın.
   - Sonucu gözlemleyin (a ve b yer değiştirecektir).

Uygulama 3: Dinamik Bellek Kullanımı

1. Kullanıcıdan bir tamsayı (n) alın.
2. new operatörünü kullanarak n elemanlı bir int dizisi için bellekten yer ayırın ve bir işaretçide adresini tutun.
3. Bu diziyi rastgele sayılarla doldurun ve ekrana yazdırın.
4. delete[] operatörünü kullanarak ayırdığınız belleği sisteme geri iade edin. Bu adımın önemini vurgulayın (bellek sızıntısını - memory leak - önlemek).

Soru & Cevap

Dr. Öğr. Üyesi Muhammed Ali KOŞAN